

01

세 가지 시각화 라이브러리 한눈에 비교

정적 그래프의 세밀한 제어부터 통계 그래프와 대화형 대시보드까지

Matplotlib, Seaborn, Plotly는 Python 데이터 시각화에서 널리 사용되지만 설계 목적이 다르다. Matplotlib은 정적 그래프의 기반과 세밀한 제어에 강하고, Seaborn은 Matplotlib 위에서 통계적 시각화를 간결하게 만든다. Plotly는 확대, 툴팁, 범례 선택 등의 상호작용을 기본 제공한다.

발음과 핵심 의미

라이브러리	한국어 발음	핵심 역할
Matplotlib	맷플롯립	기본 정적 시각화와 세밀한 사용자 정의
Seaborn	씨본	통계 데이터 시각화와 보기 좋은 기본 스타일
Plotly	플롯리	웹 기반 대화형·인터랙티브 시각화

핵심 비교

기준	Matplotlib	Seaborn	Plotly
기본 결과	정적 그래프	정적 통계 그래프	대화형 그래프
주요 데이터	list, NumPy, pandas	pandas DataFrame 중심	DataFrame, 배열, dict 등
학습 난이도	기본은 쉬우나 세부 설정이 많음	비교적 쉬움	Express는 쉬움, Objects는 중급
대표 출력	PNG, JPG, PDF, SVG	Matplotlib 출력 형식	HTML, Notebook, PNG 등
추천 상황	논문·보고서·세밀한 편집	EDA·분포·범주 비교	웹 보고서·대시보드·프레젠테이션

```
pip install matplotlib seaborn plotly pandas
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

선택의 핵심

“어느 라이브러리가 가장 좋은가?”보다 최종 산출물이 인쇄물인지, 탐색 분석인지, 웹 상호작용인지 먼저 결정한다.

02

Matplotlib

Figure와 Axes를 중심으로 그래프를 정밀하게 구성한다

Matplotlib은 Python 시각화 생태계의 기반 라이브러리이다. 공식 문서는 복잡한 그래프나 재사용할 함수에서는 **객체 지향 방식**을 권장한다. Figure는 전체 그림, Axes는 실제 데이터가 그려지는 좌표 영역이다.

기본 구조

구성 요소	설명
Figure	그래프 전체 캔버스. 여러 Axes를 포함할 수 있음
Axes	제목, x축, y축, 데이터가 배치되는 실제 그래프 영역
Axis	눈금과 눈금 라벨을 관리하는 개별 축
Artist	선, 텍스트, 범례 등 화면에 그려지는 요소

선 그래프 예제 - 객체 지향 방식

```
import matplotlib.pyplot as plt

month = [1, 2, 3, 4, 5, 6]
sales = [120, 150, 145, 190, 210, 240]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(month, sales, marker="o", color="steelblue",
        linewidth=2, label="sales")
ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Sales")
ax.grid(alpha=0.3)
ax.legend()
fig.tight_layout()
fig.savefig("sales.png", dpi=300)
plt.show()
```

장점과 한계

장점	한계
그래프의 거의 모든 요소를 세밀하게 제어할 수 있음	복잡한 그래프는 코드가 길어질 수 있음
PNG·PDF·SVG 등 정적 출력과 인쇄에 적합	대화형 웹 기능은 Plotly보다 제한적
NumPy·pandas 및 여러 과학 라이브러리와 연동	초보자는 Figure·Axes 구조가 낯설 수 있음

추천 사용

논문·보고서용 고해상도 이미지, 복잡한 서브플롯, 축·눈금·주석을 정교하게 제어해야 하는 그래프에 적합하다.

03

Seaborn

DataFrame의 변수 관계와 통계적 패턴을 간결하게 표현한다

Seaborn은 Matplotlib을 기반으로 하는 통계 시각화 라이브러리이다. pandas DataFrame과 긴밀하게 연동되며, x·y·hue·style 같은 매개변수에 열 이름을 지정하여 변수의 의미를 색상과 모양에 자동 연결한다.

대표 함수

분석 목적	함수	설명
관계	scatterplot, lineplot	두 변수 관계와 그룹별 변화
분포	histplot, kdeplot, boxplot	분포 형태·중앙값·이상값 확인
범주	barplot, countplot, violinplot	범주별 요약·빈도·분포 비교
행렬	heatmap, pairplot	상관행렬과 변수 쌍 관계
회귀	regplot, lmpplot	산점도와 회귀 추세선

그룹별 산점도와 회귀선

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
sns.set_theme(style="whitegrid")

fig, ax = plt.subplots(figsize=(7, 4))
sns.scatterplot(data=tips, x="total_bill", y="tip",
                hue="time", style="time", size="size", ax=ax)
ax.set_title("Bill and Tip by Meal Time")
plt.tight_layout()
plt.show()

# ■■■■■ ■■■ ■■
sns.regplot(data=tips, x="total_bill", y="tip")
```

Axes-level과 Figure-level

구분	예	특징
Axes-level	scatterplot, boxplot	기존 Matplotlib Axes에 결합하기 쉬움
Figure-level	relplot, catplot, displot	FacetGrid를 만들며 행·열 분할을 간단히 처리

추천 사용

탐색적 데이터 분석(EDA), 그룹별 분포 비교, 상관관계, 범주형 변수 시각화처럼 통계적 의미를 빠르게 파악할 때 적합하다.

04

Plotly

마우스 상호작용이 가능한 웹 기반 그래프를 만든다

Plotly는 확대·축소, 이동, 툴팁(hover), 범례 클릭, 이미지 저장 등의 상호작용을 기본 제공한다. **Plotly Express(px)**는 짧고 일관된 고수준 API이며, 각 함수는 수정 가능한 `graph_objects.Figure` 객체를 반환한다.

두 가지 API

API	특징	권장 상황
Plotly Express	한 번의 함수 호출로 전체 Figure 생성	일반적인 그래프, 빠른 탐색
Graph Objects	Trace와 Layout을 단계적으로 직접 구성	복잡한 혼합 그래프, 세부 제어

대화형 산점도 예제

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(
    df,
    x="sepal_width", y="sepal_length",
    color="species", size="petal_length",
    hover_data=["petal_width"],
    title="Interactive Iris Scatter Plot"
)
fig.update_layout(template="plotly_white")
fig.show()

# ■■■ HTML ■■■ ■■
fig.write_html("iris_scatter.html")
```

장점과 한계

장점	한계
툴팁·확대·범례 선택 등 상호작용을 기본 제공	대용량 데이터에서는 브라우저가 느려질 수 있음
Notebook·HTML·Dash와 연동하기 쉬움	정적 이미지 저장은 환경에 따라 추가 도구가 필요
지도·3D·애니메이션 등 다양한 고급 그래프	세밀한 구성은 Figure 구조 학습이 필요

추천 사용

사용자가 그래프를 탐색해야 하는 웹 리포트, Streamlit·Dash 대시보드, 발표용 HTML, 지도와 3D 시각화에 적합하다.

Plotly Express는 scatter, line, bar, histogram, box, violin, pie, treemap, 지도, 3D 등 다양한 그래프를 일관된 문법으로 제공한다.

05

동일한 데이터로 코드 비교

월별 매출 데이터를 세 라이브러리의 선 그래프로 표현한다

세 예제는 동일한 DataFrame을 사용한다. 핵심 차이는 데이터 전달 방식, 그룹 표현, 결과 객체, 상호작용이다.

```
import pandas as pd

df = pd.DataFrame({
    "month": [1, 2, 3, 4, 5, 6],
    "sales": [120, 150, 145, 190, 210, 240],
    "target": [130, 140, 160, 180, 200, 220]
})
```

Matplotlib - 세밀한 제어

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(df["month"], df["sales"], marker="o", label="sales")
ax.plot(df["month"], df["target"], linestyle="--", label="target")
ax.set(xlabel="Month", ylabel="Amount", title="Sales vs Target")
ax.legend(); ax.grid(alpha=.3); plt.show()
```

Seaborn - DataFrame 중심 통계 표현

```
long_df = df.melt("month", var_name="type", value_name="amount")
sns.lineplot(data=long_df, x="month", y="amount",
             hue="type", style="type", markers=True)
plt.title("Sales vs Target"); plt.show()
```

Plotly - 기본 대화형 기능

```
fig = px.line(long_df, x="month", y="amount", color="type",
             markers=True, title="Sales vs Target")
fig.show()
```

코드 해석

관점	Matplotlib	Seaborn	Plotly
결과 객체	Figure, Axes	Axes 또는 Grid	plotly Figure
그룹 표현	각 선을 직접 추가	hue/style에 열 지정	color/symbol에 열 지정
상호작용	기본 정적	기본 정적	hover·zoom 기본 제공

06

선택 기준과 종합 정리

목적 · 산출물 · 사용자 경험에 맞춰 조합한다

실무에서는 하나만 고집하지 않고 단계에 따라 조합하는 경우가 많다. Seaborn으로 빠르게 패턴을 탐색하고, Matplotlib으로 보고서용 그래프를 정교하게 다듬거나, Plotly로 대화형 결과를 제공할 수 있다.

상황별 추천

상황	1순위	이유
논문·인쇄 보고서	Matplotlib	고해상도 정적 출력과 세밀한 편집
빠른 탐색 분석(EDA)	Seaborn	통계 그래프와 그룹 비교가 간결함
웹 대시보드	Plotly	툴팁·확대·범례 선택 등 상호작용
상관행렬·분포 비교	Seaborn	heatmap·pairplot·violinplot 지원
복잡한 서브플롯	Matplotlib	Figure·Axes를 직접 정교하게 구성
지도·3D·애니메이션	Plotly	웹 기반 고급 상호작용 지원

최종 비교 평가

평가 항목	Matplotlib	Seaborn	Plotly
코드 간결성	보통	높음	Express는 높음
세밀한 정적 편집	매우 높음	높음(필요 시 Matplotlib 사용)	높음
통계 시각화	보통	매우 높음	높음
웹 상호작용	낮음	낮음	매우 높음
인쇄·논문 출력	매우 적합	적합	정적 변환 시 적합

선택 체크리스트

☒ 최종 결과가 정적 이미지인가, 대화형 HTML인가? ☒ 통계 분포와 범주 비교가 핵심인가? ☒ 축·주석·레이아웃을 얼마나 세밀하게 수정해야 하는가? ☒ 데이터 크기와 실행 환경은 무엇인가? ☒ 결과물을 Notebook, PDF, 웹 앱 중 어디에 배포하는가?

한 문장 정리

Matplotlib은 “정밀한 기본 도구”, Seaborn은 “통계 분석을 위한 편리한 도구”, Plotly는 “사용자가 탐색하는 대화형 도구”이다.

참고 문서

Matplotlib: <https://matplotlib.org/stable/>

Seaborn: <https://seaborn.pydata.org/>

Plotly Python: <https://plotly.com/python/>